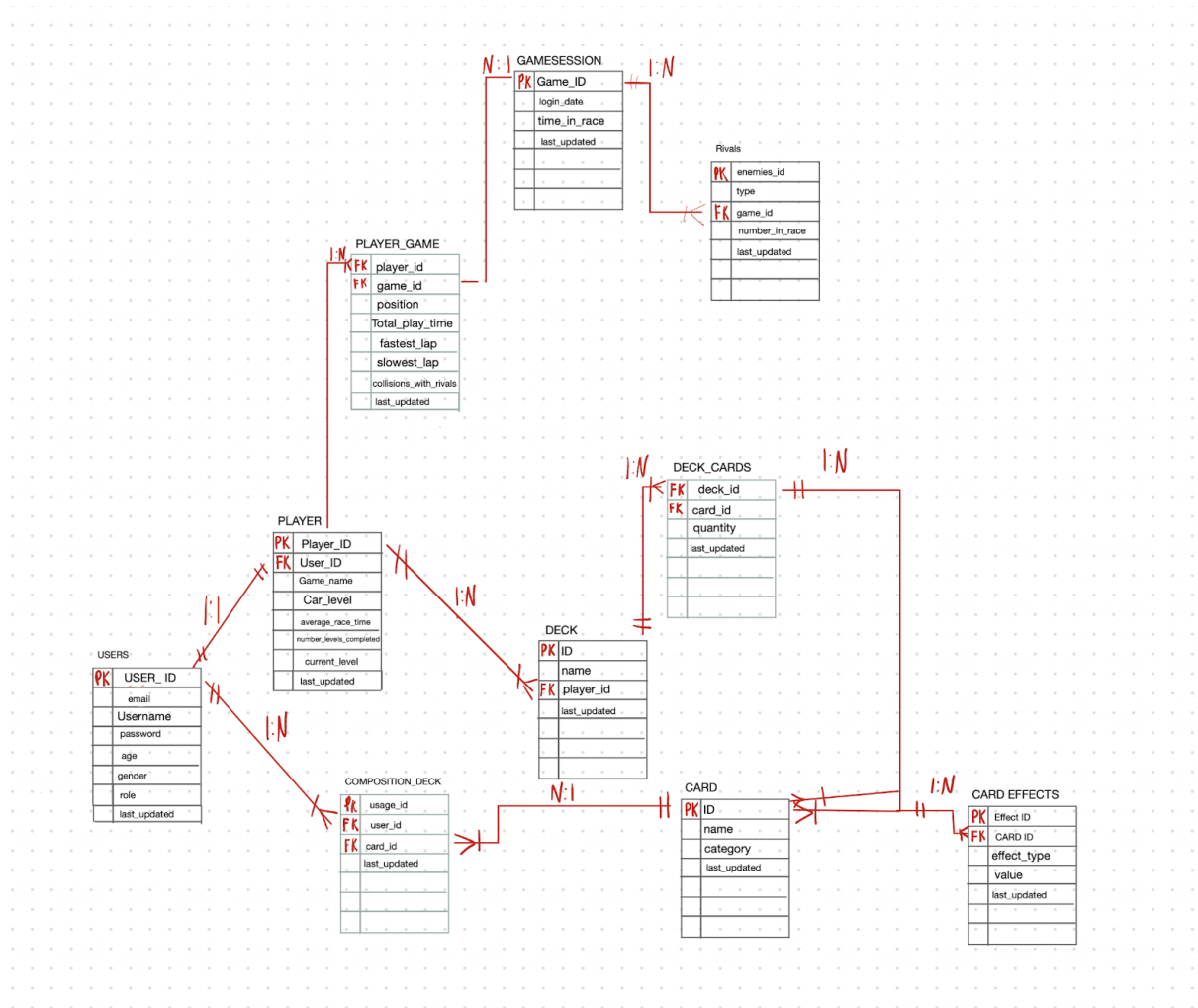


Diagram:



Possible integrity constraints:

Tabla	Atributo	Constraint
USERS	user_id	PRIMARY KEY, AUTO_INCREMENT
	email	UNIQUE, NOT NULL
	age	CHECK (age > 0 AND age < 120)
PLAYER	player_id	PRIMARY KEY
	user_id	UNIQUE, FOREIGN KEY
	car_level	DEFAULT 1, CHECK (car_level >= 1)
GAMESESSION	game_id	PRIMARY KEY
	time_in_race	CHECK (time_in_race > 0)

PLAYER_GAME	player_id	PRIMARY KEY (FK)
	game_id	PRIMARY KEY (FK)
	position	CHECK (position > 0)
CARD	card_id	PRIMARY KEY
	name	UNIQUE, NOT NULL
CARD_EFFECTS	value	CHECK (value >= 0)

The final conceptual schema satisfies the Third Normal Form (3NF). Making sure to fulfill the three normal forms along the way.

Within the schema, we ensured 1NF by ensuring that all attributes contain only atomic values, with no repeating groups. With each table having a primary key, multivalued data stores as independent entities and a specific table for **CARD EFFECTS**. For 2NF, we ensured that every non-key attribute depends on the entire primary key. For example, by solving the many-to-many relationships by creating associative entities like **PLAYER_GAME** and **DECK_CARDS**.

And lastly, for the 3NF, we focused on eliminating transitive dependencies. User information is separated from player information, deck information is separated from card information, card effects are separated from cards, and the information for rivals is separated from game sessions.

These design choices were implemented to minimize redundancy.

Explanation for keys:

The selection of primary keys and foreign keys was based on logical relationships between the entities, in accordance with standard SQL database design principles.

Justification for each table:

USERS.

Primary KEY:

- User_ID
Each user account must have a specific id that's unique.

PLAYER.

Primary KEY:

- Player_ID

Foreign KEY:

- User_ID -> USERS (USER_ID)

A player profile belongs to a user account. The foreign key prevents player records from existing without an associated user, creating referential integrity.

GAMESESSION.

Primary KEY:

- Game_ID

This attribute maintains each game session's uniqueness.

RIVALS.

Primary KEY:

- enemies_id

Foreign KEY:

- game_id -> GAMESESSION (Game_ID)

These keys ensure that rivals exist only within a specific game session. Each rival record is associated with the session they appear in. This allows flexibility between game sessions.

PLAYER_GAME.

Composite Primary Key::

- (player_id, game_id)

Foreign KEY:

- player_id->PLAYER(Player_ID)
- game_id->GAMESESSION(Game_ID)

In this table we created a composite key that resolves many-to-many relationships. It guarantees that the same player cannot appear twice in the same game session.

DECK.

Primary KEY:

- ID

Foreign KEY:

- player_id -> PLAYER (Player_ID)

Each deck belongs to a specific player. But these keys allow a player to own multiple decks.

CARD.

Primary KEY:

- ID

Each card has a unique id for authentication.

DECK_CARDS.

Composite Primary KEY:

- (deck_id, card_id)

Foreign KEY:

- deck_id-> DECK(ID)
- card_id-> CARD (ID)

This table tackles the many-to-many relationship between decks and cards. The justification for these keys is to allow a deck to contain many cards and allow a card to appear in many decks.

CARD_EFFECTS.

Primary KEY:

- Effect_ID

Foreign KEY:

- CARD_ID -> CARD(ID)

A card may have multiple effects, while each effect belongs to one card.

COMPOSITION_DECK.

Primary KEY:

- usage_id

Foreign KEY:

- user_ID -> USERS (USER_ID)
- card_id -> CARD(ID)

This table tracks which cards are associated with a user. The justification for these keys is based on how it allows tracking deck composition/card usage which becomes important for game statistics and game balancing. The primary key identifies each record.